46-926, Spring 2019: Homework #2

Due 5PM, February 4, 2019.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

1. *Prediction error and dimension.* In your last homework, you explored the role of dimension in the prediction error of linear regression. In this problem, you will extend that simulation to include the lasso.

   Note: You can use either python or R for this problem. There are some hints at the end of this problem that are language dependent. Please read them first.

   Carry out essentially the same regression simulation as last homework (feel free to modify the same code, or the code from the solutions when they come out), but this time you will consider two different predictors:

   (a) The linear regression you did last time

   (b) Lasso regression, with regularization parameter $\sqrt{\frac{2 \log p}{n}}$.

   IMPORTANT: Change the true generating model to be $Y = \frac{1}{\sqrt{2}} X_1 + \varepsilon$ where ε has standard deviation $\frac{1}{\sqrt{2}}$, which is different from the previous homework. This is to hide a minor, stupid difference between R and python that I'll mention in class.

   Each time through the inner loop (or equivalent) of your simulation, you will fit both models, and compute the average squared prediction error for both estimates.

   Just like last time, use $n = 100$ and average 100 samples for each size. Just do $p = 2, \ldots, 80$, $p = 1$ will lead to a bit of odd behavior.

   Plot both of the average squared prediction errors against p (on the same plot). Make sure to use colors or symbols to distinguish between the models, and add a legend.

   What is the conclusion of your experiment – how do the methods compare?

Hints:

If you want to use `python`, use the `Lasso` function from `sklearn.linear_model`. This is called like

```
model = Lasso(alpha=[What you want], normalize=True,
              fit_intercept=True)
model.fit(X_train, y_train)
#Then call model.predict, etc.
```

If you want to use `R`, use `glmnet` from the `glmnet` package. This will very closely follow the example from class.

2. In this problem, we will explore ideas related to a recent paper, Preis et al. (2013), which attempts to model DJIA returns based on Google Trends data.

   In `trends_train.csv` and `trends_test.csv`, you will find training and test data sets of matching column format. For now we'll work with the training data set.

   The data sets consist of the following columns:

   - The date of the Sunday that *ends* the week of google trends data, which is aggregated Monday - Sunday. The volume differences in the other columns start one week before this date and end on this date.
   - Google trends data for 96 words, with words selected by the methodology in the paper. The time series for each word reflects *change in* the volume of search queries that week, compared to the previous week. The original time series (before taking differences) were normalized to have a maximum of 100 over time; you can find these in `trend_timeseries.csv` just for fun.
   - The weekly DJIA log returns for the week *after* the google trends data (Monday - Monday adjusted closing price).

   You are welcome to do this problem in either R or python, but I highly recommend R. In python, I think you would have to write your own 1 s.e. rule. (All the information to do so is available in the model though.)

   Preis, T., Moat, H. S., & Stanley, H. E. (2013). Quantifying trading behavior in financial markets using Google Trends. Scientific reports, 3, srep01684.

   (a) Load the training data and see what you have. Fit the lasso to predict the weekly log return from the weekly changes in search volume. Use the defaut range of penalization parameters. Plot the coefficients as a function of $\log(\lambda)$.

(b) Carry out cross-validation for the lasso (using the built in procedure). Plot the cross-validation curve with its standard errors.

Note: For the purpose of this problem, we're going to carry out cross-validation as if the samples were i.i.d., neglecting the temporal structure. In reality, this would be something to worry about.

(c) Look at the models that you obtain through each selection rule (minimization vs. 1 s.e.). How many nonzero coefficients does each model have (other than the intercept)? What does this suggest to you about the amount of signal in the problem?

Note: The output of cross-validation depends on the random splits that you select. In this problem, you should get different values for $\lambda_{\min}$ *and* $\lambda_{1\text{s.e.}}$ *to make it interesting. If by some chance you select the same model both ways, just rerun cross-validation again or change your seed.*

(d) Let's try the $\lambda_{\min}$ model out. Make predictions on your training data by using the predict function and passing your training X in as the `newx` parameter. The `s` parameter is the λ value you want to fit at. Make a scatterplot of your fitted values against the true values; is there any pattern?

(e) We could carry out a simple (and overly simplified) trading strategy based on your model, as outlined in the paper. Each Sunday, we look at the prediction from your model. If the log returns for the next week are predicted to be positive, we buy on the Monday and sell on the following Monday. If the log returns are predicted to be negative, we do the opposite. This lets us realize log returns for each week of

$$(\text{sign of our prediction}) \cdot (\text{DJIA log return})$$

which is easy to work with. Compute your return for this strategy on the training data, using your $\lambda_{\min}$ model. Compare it to the return you would have realized if you bought on the first day of the data set and sold on the last day.

(f) Now apply your $\lambda_{\min}$ model, trained on the training set, to your test data. Again, you just use the predict function, changing the `newx` parameter. Plot your fitted values as before.

Carry out the same simple strategy on the test data. How well do you do? Again, compare to the overall return for the DJIA in this period. In the end, would you have been better off with the $\lambda_{1\text{s.e.}}$ model?

3. **Fun with the SVD!** The *Singular Value Decomposition* (SVD) is very useful for understanding many objects in machine learning. In this problem, you will see that it gives insight into ridge regression (and linear regression). We will see it appear again throughout the semester.

   Let $X \in \mathbb{R}^{n \times p}$ be our data matrix. For this problem, we assume that $n > p$ and that X is full column rank (though this is not necessary for SVD). We can write

   $$X = UDV^T$$

   where $U \in \mathbb{R}^{n \times p}$ and $V \in \mathbb{R}^{p \times p}$ are orthogonal and $D \in \mathbb{R}^{p \times p}$ is diagonal. (You may have seen the SVD written with different dimensions in the past. Writing it like this, with U being non-square instead of D, is sometimes called the "thin" version of the SVD. Everything for this problem will work out a little prettier with this version.) Let $d_1, \ldots, d_p$ be the p diagonal elements of D.

   Note: There are many hints on linear algebra at the end of this problem.

   (a) Consider least squares linear regression. We know that $\widehat{Y} = X\widehat{\beta} = X(X^TX)^{-1}X^TY$. Replace X with its SVD, and simplify as much as possible. How can you interpret this in terms of the columns of U and/or V? (see hints)

   (Fun ungraded question to think about: if you already knew the SVD of X, what is the computation time to run a regression?)

   (b) Now consider ridge regression with fixed λ. We showed last homework that

   $$\widehat{Y}_{ridge} = X\widehat{\beta}_{ridge} = X(X^TX + \lambda I)^{-1}X^TY.$$

   Again, replace X with its SVD and simplify as much as possible. You will wind up with a matrix product that looks more complicated than in (a).

   Hints:

   - You will find it very useful to to replace I with VV^T at some point. This will let you factor and invert, even though it doesn't feel like a simplification.
   - You will know when you are done simplifying because your final answer will have Us and Ds, but no Vs.

(c) Try rewriting the product from (b) as a sum. You should be able to obtain a sum of the form:

$$\widehat{Y}_{ridge} = \sum_{j=1}^{p} (\textbf{A scalar you must find}) \cdot u_j u_j^T y$$

where u_j is the j^{th} column of U. Compare this to (b):

- If $\lambda = 0$, do you get the same answer as linear regression (part b)?
- If $\lambda > 0$, what is the effect of the scalar you had to find? Which terms does lambda shrink the most, those with large d_j or those with small d_j? How would you interpret that in terms of the amount of variance in each direction? (See PCA hint below.)

Hints for Problem 3:

- By orthogonality, $U^T U = V^T V = I_p$, the $p \times p$ identity matrix. Also, $VV^T = I_p$.
- D is a diagonal matrix, so $D^T = D$ and $DD = D^2$ is just a diagonal matrix with $d_1^2, \ldots, d_p^2$ on the diagonal.
- UU^T is *not* the identity. What is it? (Hints: it looks a lot like quantities that appeared in our PCA slides, think in terms of the columns of U).
- It turns out that the columns of V are the PCA directions for X, that $\frac{1}{n} d_j^2$ is the variance in the v_j direction, and that the i^{th} entry of u_j (column) is the (normalized) length of the projection of x_i onto v_j. We will see that important regression quantities can be expressed in terms of these quantities.
- If any square matrices $A, B \in \mathbb{R}^{p \times p}$ are invertable, then $(AB)^{-1} = B^{-1} A^{-1}$. In particular, for V above and D an invertable matrix, $(VDV^T)^{-1} = VD^{-1}V^T$ (since V^T is the inverse of V, and vice versa).
- If D were a diagonal matrix, then $UDU^T Y = \sum_{i=1}^{n} D_{ii} u_i u_i^T y$.